

Phase 3: Binary Tree Representation and Construction

0. Purpose of This Phase

Phase 3 teaches how a binary tree is stored and created in computer memory.

In Phase 1, you learned what a tree is.

In Phase 2, you learned the mathematics of binary tree shapes.

Now, in Phase 3, you learn this important question:

I can draw a binary tree on paper, but how does the computer store it?

There are two main ways to represent a binary tree:

1. **Array representation**
2. **Linked representation**

The main beginner idea is:

Array representation stores nodes in positions.

Linked representation stores nodes using pointers.

Queue-based construction builds the tree level by level.

1. Current Progress Analysis

Before Phase 3, you should already know these ideas:

- A tree is made of nodes and edges.
- A binary tree is a tree where each node can have at most two children.
- The two child positions are called the left child and right child.
- A leaf node has no children.
- An internal node has at least one child.
- Height counts edges from the root to the deepest node.
- A binary tree can have many possible shapes.
- Pointers can connect one node to another node.

So your current position is:

You already understand what binary trees look like.
Now you are learning how to store and create them in C/C++ memory.

This phase is important because later phases, especially traversal, need an actual tree structure to work on.

2. What Phase 3 Covers

Phase 3 covers these main topics:

1. Array representation of binary trees
 2. Array index formulas
 3. Linked representation of binary trees
 4. Node structure with left and right child pointers
 5. Queue-based tree construction
 6. Why the queue stores `Node*` addresses
 7. Using `-1` as a null child marker
 8. Dry run of tree creation
 9. Complete C++ program for building and printing a binary tree
 10. Common beginner mistakes
-

3. Two Ways to Store a Binary Tree

A binary tree can be stored in two major ways.

Method	Simple Meaning
Array representation	Store nodes in array positions
Linked representation	Store nodes separately and connect them using pointers

Simple memory trick:

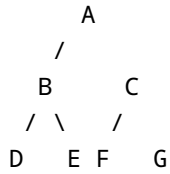
Array representation = positions
Linked representation = pointers

4. Array Representation

4.1 What Is Array Representation?

In array representation, we store the binary tree level by level from left to right.

Consider this binary tree:



We store it in an array like this:

Index:	1	2	3	4	5	6	7
Node:	A	B	C	D	E	F	G

Important:

In this curriculum, array representation starts from index 1.

So index 0 is ignored.

```
tree[1] = A
tree[2] = B
tree[3] = C
tree[4] = D
tree[5] = E
tree[6] = F
tree[7] = G
```

5. Why Index 1 Is Used

The formulas are very simple when the root starts at index 1.

If a node is at index i , then:

```
Left child index  = 2 * i
Right child index = 2 * i + 1
Parent index      = i / 2
```

These formulas are the main reason index 1 is used.

If we started from index 0, the formulas would be different.

6. Array Index Formulas

6.1 Formula Table

Relationship	Formula
Left child of node at index <code>i</code>	<code>2 * i</code>
Right child of node at index <code>i</code>	<code>2 * i + 1</code>
Parent of node at index <code>i</code>	<code>i / 2</code>

Integer division is used for the parent formula.

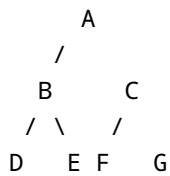
For example:

$$7 / 2 = 3$$

So the parent of index 7 is index 3.

7. Array Formula Example

Use this tree:



Array form:

Index:	1	2	3	4	5	6	7
Node:	A	B	C	D	E	F	G

Node `C` is at index 3.

Left Child of C

```
Left child = 2 * i  
            = 2 * 3  
            = 6
```

Index 6 contains **F**.

So:

```
Left child of C = F
```

Right Child of C

```
Right child = 2 * i + 1  
             = 2 * 3 + 1  
             = 7
```

Index 7 contains **G**.

So:

```
Right child of C = G
```

Parent of G

Node **G** is at index 7.

```
Parent = i / 2  
        = 7 / 2  
        = 3
```

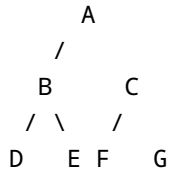
Index 3 contains **C**.

So:

```
Parent of G = C
```

8. Dry Run: Array Representation of A to G

Tree:



Array:

Index:	1	2	3	4	5	6	7
Node:	A	B	C	D	E	F	G

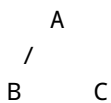
Node	Index	Left Child Index	Right Child Index
A	1	2 = B	3 = C
B	2	4 = D	5 = E
C	3	6 = F	7 = G
D	4	8	9
E	5	10	11
F	6	12	13
G	7	14	15

D, E, F, and G are leaf nodes, so their child positions are empty.

9. When Array Representation Is Useful

Array representation is best when the tree is complete or almost complete.

A complete-looking tree stores nicely in an array:



```

      / \ /
     D  E F G

```

Array:

Index:	1	2	3	4	5	6	7
Node:	A	B	C	D	E	F	G

There are no empty gaps.

So array representation is clean and efficient here.

10. Problem With Array Representation

Array representation can waste space when the tree is sparse.

Example:

```

      A
     /
    B
   /
  C

```

Using index formulas:

A is at index 1
 B is left child of A, so B is at index 2
 C is left child of B, so C is at index 4

Array:

Index:	1	2	3	4
Node:	A	B	-	C

Index 3 is empty.

If the tree becomes very skewed, many array positions may be wasted.

So:

Array representation is simple, but it may waste memory for sparse or skewed trees.

11. Array Representation Code

```
#include <iostream>
#include <vector>
using namespace std;

class ArrayBinaryTree {
private:
    vector<char> tree;

public:
    ArrayBinaryTree(int size) {
        tree.resize(size + 1, '-'); // index 0 is ignored
    }

    void setNode(int index, char value) {
        tree[index] = value;
    }

    int left(int i) {
        return 2 * i;
    }

    int right(int i) {
        return 2 * i + 1;
    }

    int parent(int i) {
        return i / 2;
    }

    char at(int i) {
        return tree[i];
    }
};

int main() {
    ArrayBinaryTree tree(7);

    tree.setNode(1, 'A');
    tree.setNode(2, 'B');
    tree.setNode(3, 'C');
    tree.setNode(4, 'D');
```



```

    tree.setNode(5, 'E');
    tree.setNode(6, 'F');
    tree.setNode(7, 'G');

    cout << "Left child of C: " << tree.at(tree.left(3)) << endl;
    cout << "Right child of C: " << tree.at(tree.right(3)) << endl;
    cout << "Parent of G: " << tree.at(tree.parent(7)) << endl;

    return 0;
}

```

Expected Output

```

Left child of C: F
Right child of C: G
Parent of G: C

```

12. Explanation of the Array Code

This line creates the array:

```
tree.resize(size + 1, '-');
```

We use `size + 1` because index 0 is ignored.

If the tree needs 7 positions, we create 8 spaces:

```
0 1 2 3 4 5 6 7
```

Index 0 is not used.

This function stores a node:

```

void setNode(int index, char value) {
    tree[index] = value;
}

```

Example:

```
tree.setNode(3, 'C');
```

This means:

Put C at index 3.

This function gives the left child index:

```
int left(int i) {  
    return 2 * i;  
}
```

This function gives the right child index:

```
int right(int i) {  
    return 2 * i + 1;  
}
```

This function gives the parent index:

```
int parent(int i) {  
    return i / 2;  
}
```

This function returns the node value:

```
char at(int i) {  
    return tree[i];  
}
```

13. Linked Representation

13.1 What Is Linked Representation?

In linked representation, each node is created separately in memory.

Then nodes are connected using pointers.

A binary tree node stores three things:

1. Data
2. Address of the left child
3. Address of the right child

So one node looks like this:

```
+-----+-----+-----+
| data   | lchild  | rchild  |
+-----+-----+-----+
```

Simple meaning:

The node stores its value and the addresses of its children.

14. Linked Node Structure

```
struct Node {
    int data;
    Node* lchild;
    Node* rchild;

    Node(int value) {
        data = value;
        lchild = nullptr;
        rchild = nullptr;
    }
};
```

Explanation

```
int data;
```

This stores the value inside the node.

Example:

```
10, 20, 30, etc.
```

```
Node* lchild;
```

This stores the address of the left child.

```
Node* rchild;
```

This stores the address of the right child.

```
lchild = nullptr;  
rchild = nullptr;
```

This means the node starts with no children.

15. What Is `nullptr`?

`nullptr` means the pointer is not pointing to any node.

Example:

```
lchild = nullptr;
```

This means:

```
There is no left child yet.
```

```
rchild = nullptr;
```

This means:

```
There is no right child yet.
```

This is very important.

If we do not initialize pointers to `nullptr`, they may contain garbage addresses.

Garbage addresses can make a program crash.

16. Creating a Linked Binary Tree Manually

Suppose we want this tree:

```
      10
     /
    20   30
```

We can create it like this:

```
Node* root = new Node(10);
root->lchild = new Node(20);
root->rchild = new Node(30);
```

Step-by-Step Meaning

```
Node* root = new Node(10);
```

This creates node 10.

```
10
```

`root` stores the address of node 10.

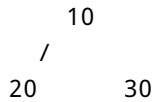
```
root->lchild = new Node(20);
```

This creates node 20 and connects it as the left child of 10.

```
      10
     /
    20
```

```
root->rchild = new Node(30);
```

This creates node 30 and connects it as the right child of 10.

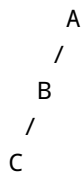


17. Why Linked Representation Is Common

Linked representation is common because it does not waste as much memory for sparse trees.

Each node only stores pointers to the children it actually has.

Example:



In linked representation:

- A points to B.
- B points to C.
- Missing children are simply `nullptr`.

No large empty array positions are needed.

18. Array vs Linked Representation

Feature	Array Representation	Linked Representation
How nodes are stored	In array positions	As separate nodes in memory
How children are found	By formulas	By pointers
Main formulas	<code>2*i</code> , <code>2*i+1</code> , <code>i/2</code>	No index formulas needed
Best for	Complete or almost-complete trees	General binary trees
Can waste space?	Yes, if sparse	Less likely

Feature	Array Representation	Linked Representation
Uses pointers?	Not required	Yes

Simple memory trick:

Array = calculate child positions.

Linked = follow child pointers.

19. Why We Need a Queue to Build a Tree

Manually creating every node is possible, but it is not convenient for large trees.

Example:

```
root->lchild = new Node(20);
root->rchild = new Node(30);
root->lchild->lchild = new Node(40);
root->lchild->rchild = new Node(50);
```

This becomes hard to manage.

Instead, we can build the tree level by level using a queue.

A queue is useful because it follows this rule:

First In, First Out

Short form:

FIFO

A queue is like a line of people.

The first person who enters the line is the first person served.

20. Queue-Based Construction Idea

The queue stores nodes whose children still need to be created.

The process is:

1. Create the root.
2. Put the root into the queue.
3. Take one node out of the queue.
4. Create its left child if it exists.
5. Create its right child if it exists.
6. Put any newly created children into the queue.
7. Repeat until the queue is empty.

This builds the tree level by level.

21. Why the Queue Stores `Node*`

The queue must store addresses of nodes.

Correct:

```
queue<Node*> q;
```

Wrong:

```
queue<int> q;
```

Why is `queue<int>` wrong?

Because an integer only stores the value.

Example:

```
20
```

But to attach children, we need the actual node in memory.

We need to do this:

```
p->lchild = new Node(value);  
p->rchild = new Node(value);
```


That means `p` must be a node pointer.

So the queue must store `Node*`.

Simple memory trick:

Values cannot grow children.
Node addresses can grow children.

22. Meaning of `-1` in Input

In this phase, `-1` means:

No child exists here.

Example:

20 -1 50

This could mean:

For node 20:
left child does not exist
right child is 50

Important:

`-1` is not stored as a node.
It is only a signal meaning null child.

So if input value is `-1`, we do not create a new node.

23. Queue Construction Algorithm

1. Read the root value.
2. If the root value is `-1`, the tree is empty.
3. Otherwise, create the root node.
4. Push the root node address into the queue.

5. While the queue is not empty:
 - a. Remove the front node from the queue.
 - b. Read its left child value.
 - If value is not -1, create the left child.
 - Push the left child address into the queue.
 - c. Read its right child value.
 - If value is not -1, create the right child.
 - Push the right child address into the queue.
6. Stop when the queue becomes empty.

24. Dry Run: Building a Binary Tree

Input:

```
10 20 30 40 50 60 70 -1 -1 -1 -1 -1 -1 -1 -1
```

This input means:

- Root is 10.
- Children of 10 are 20 and 30.
- Children of 20 are 40 and 50.
- Children of 30 are 60 and 70.
- Children of 40, 50, 60, and 70 do not exist.

25. Dry Run Step 1: Create Root

First value:

```
10
```

Create root node:

```
10
```

Queue:

```
[10]
```

Meaning:

Node 10 is waiting to receive its children.

26. Dry Run Step 2: Process Node 10

Remove 10 from the queue.

Read two values:

20 30

So:

20 is left child of 10
30 is right child of 10

Tree:

```
      10
     /
    20   30
```

Queue:

[20, 30]

Meaning:

Nodes 20 and 30 are waiting to receive their children.

27. Dry Run Step 3: Process Node 20

Remove 20 from the queue.

Read two values:

40 50

So:

40 is left child of 20
50 is right child of 20

Tree:

```
      10
     /
    20   30
   /
  40   50
```

Queue:

[30, 40, 50]

28. Dry Run Step 4: Process Node 30

Remove 30 from the queue.

Read two values:

60 70

So:

60 is left child of 30
70 is right child of 30

Tree:

```
      10
     /
    20   30
   /
  40   50
```

/ \ /
40 50 60 70

Queue:

[40, 50, 60, 70]

29. Dry Run Step 5: Process Leaf Nodes

Now we process 40.

Read:

-1 -1

So 40 has no children.

Then process 50.

Read:

-1 -1

So 50 has no children.

Then process 60.

Read:

-1 -1

So 60 has no children.

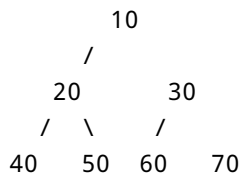
Then process 70.

Read:

-1 -1

So 70 has no children.

Final tree:



Final queue:

[]

The queue is empty, so construction is finished.

30. Level-Order Output

If we print the final tree level by level, we get:

10 20 30 40 50 60 70

This matches the input order before the `-1` values.

31. Complete C++ Program: Build Tree from Level Input

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

struct Node {
    int data;
    Node* lchild;
```

```

Node* rchild;

Node(int value) {
    data = value;
    lchild = nullptr;
    rchild = nullptr;
}

};

Node* createTreeFromLevelInput(const vector<int>& input) {
    if (input.empty() || input[0] == -1) {
        return nullptr;
    }

    int pos = 0;

    Node* root = new Node(input[pos]);
    pos++;

    queue<Node*> q;
    q.push(root);

    while (!q.empty() && pos < input.size()) {
        Node* p = q.front();
        q.pop();

        // Create left child if it exists
        if (pos < input.size() && input[pos] != -1) {
            p->lchild = new Node(input[pos]);
            q.push(p->lchild);
        }
        pos++;

        // Create right child if it exists
        if (pos < input.size() && input[pos] != -1) {
            p->rchild = new Node(input[pos]);
            q.push(p->rchild);
        }
        pos++;
    }

    return root;
}

void printLevelOrder(Node* root) {
    if (root == nullptr) {
        return;
    }
}

```

```

queue<Node*> q;
q.push(root);

while (!q.empty()) {
    Node* p = q.front();
    q.pop();

    cout << p->data << " ";

    if (p->lchild != nullptr) {
        q.push(p->lchild);
    }

    if (p->rchild != nullptr) {
        q.push(p->rchild);
    }
}

int main() {
    vector<int> input = {
        10, 20, 30, 40, 50, 60, 70,
        -1, -1, -1, -1, -1, -1, -1, -1
    };

    Node* root = createTreeFromLevelInput(input);

    cout << "Level order output: ";
    printLevelOrder(root);
    cout << endl;

    return 0;
}

```

Expected Output

```
Level order output: 10 20 30 40 50 60 70
```


32. Explanation of the Complete Program

Header Files

```
#include <iostream>
#include <vector>
#include <queue>
```

`iostream` is used for input and output.

`vector` is used to store the input values.

`queue` is used to build and print the tree level by level.

Node Structure

```
struct Node {
    int data;
    Node* lchild;
    Node* rchild;
};
```

Each node stores:

- data
 - left child pointer
 - right child pointer
-

Constructor

```
Node(int value) {
    data = value;
    lchild = nullptr;
    rchild = nullptr;
}
```

This creates a node with a value.

At first, the node has no children.

Empty Tree Check

```
if (input.empty() || input[0] == -1) {  
    return nullptr;  
}
```

If there is no input, there is no tree.

If the first value is `-1`, the root does not exist.

So the tree is empty.

Root Creation

```
Node* root = new Node(input[pos]);  
pos++;
```

This creates the root node from the first input value.

Queue Creation

```
queue<Node*> q;  
q.push(root);
```

The root is pushed into the queue because we still need to create its children.

Processing the Queue

```
while (!q.empty() && pos < input.size()) {
```

This loop continues while:

- the queue still has nodes waiting for children
 - there are still input values left
-

Getting the Current Parent

```
Node* p = q.front();  
q.pop();
```

The front node becomes the current parent.

We remove it from the queue because we are now processing it.

Creating the Left Child

```
if (pos < input.size() && input[pos] != -1) {  
    p->lchild = new Node(input[pos]);  
    q.push(p->lchild);  
}  
pos++;
```

If the current input is not `-1`, create a left child.

Then push that child into the queue because it may also have children later.

Creating the Right Child

```
if (pos < input.size() && input[pos] != -1) {  
    p->rchild = new Node(input[pos]);  
    q.push(p->rchild);  
}  
pos++;
```

If the current input is not `-1`, create a right child.

Then push that child into the queue.

33. Why `printLevelOrder` Also Uses a Queue

The function `printLevelOrder` prints the tree level by level.

That means it also needs a queue.

```

void printLevelOrder(Node* root) {
    if (root == nullptr) {
        return;
    }

    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        Node* p = q.front();
        q.pop();

        cout << p->data << " ";

        if (p->lchild != nullptr) {
            q.push(p->lchild);
        }

        if (p->rchild != nullptr) {
            q.push(p->rchild);
        }
    }
}

```

This prints:

1. Root
2. Root's children
3. Root's grandchildren
4. And so on

That is why the output is:

```
10 20 30 40 50 60 70
```

34. C Queue Design Note

In C, we do not have `std::queue` like in C++.

So the original lesson uses the idea of a custom queue header.

Important idea:

The queue should store `struct Node*`, not plain integers.

Why?

Because each queue element must point to an actual tree node.

A C-style queue may be implemented as a circular queue.

Conceptually, it stores this:

```
struct Node* queueArray[size];
```

Each slot stores the address of a node.

35. C++ Queue Class Design Note

The C++ lesson also discusses a class-based queue idea.

One possible design uses `Node**`.

This may look confusing at first.

`Node*` means:

Pointer to one Node

`Node**` means:

Pointer to an array of Node pointers

So if we write:

```
Node** Q;
```

It means:

Q is an array where each element stores a `Node*`.

For beginner practice, modern C++ makes this easier:

```
queue<Node*> q;
```

This is simpler and safer.

36. Time and Space Complexity

Array Representation

Finding child or parent index takes constant time.

```
left  = 2*i  
right = 2*i + 1  
parent = i/2
```

So each navigation operation is:

```
O(1)
```

Space depends on the size of the array.

If the array has n positions, space is:

```
O(n)
```

But for sparse trees, some positions may be empty.

Linked Representation

Creating each node takes constant time.

If there are n nodes, total node space is:

```
O(n)
```

Queue-based construction visits each input node once.

So time is:

$O(n)$

The queue may hold many nodes at once.

So queue space can be:

$O(n)$

37. Common Beginner Mistakes

Mistake 1: Starting Array Formulas at Index 0

Wrong idea:

Root is at index 0, left child = $2*i$

This is wrong for this curriculum.

Correct idea:

Root is at index 1.
Left child = $2*i$.
Right child = $2*i + 1$.
Parent = $i/2$.

If you start at index 0, you need different formulas.

Mistake 2: Putting Data Values into the Queue

Wrong:

```
queue<int> q;
```

Correct:

```
queue<Node*> q;
```

Why?

Because we need to attach children to real nodes.

A value like `20` cannot have children.

A node pointer can have children:

```
p->lchild = new Node(40);
```

Mistake 3: Forgetting to Push Newly Created Children

Wrong idea:

```
p->lchild = new Node(value);
```

But then forgetting:

```
q.push(p->lchild);
```

If you do not push the new child, the program will never ask for that child's children.

Correct:

```
p->lchild = new Node(value);  
q.push(p->lchild);
```

Mistake 4: Treating `-1` as a Real Node

Wrong:

```
p->lchild = new Node(-1);
```

Correct:


```
if (input[pos] != -1) {  
    p->lchild = new Node(input[pos]);  
}
```

-1 means no child.

It should not become a node.

Mistake 5: Forgetting `nullptr`

Bad constructor:

```
Node(int value) {  
    data = value;  
}
```

Problem:

`lchild` and `rchild` are not initialized.

Correct constructor:

```
Node(int value) {  
    data = value;  
    lchild = nullptr;  
    rchild = nullptr;  
}
```

This means the node starts safely with no children.

Mistake 6: Thinking Array and Linked Representation Are the Same

They store the same tree idea, but in different ways.

Array representation uses positions.

Linked representation uses addresses.

38. Key Vocabulary Table

Term	Simple Meaning
Representation	How data is stored in memory
Array representation	Store tree nodes in array positions
Linked representation	Store nodes using pointers
Index	Position in an array
Root index	Usually 1 in this curriculum
Left child formula	$2*i$
Right child formula	$2*i + 1$
Parent formula	$i/2$
Pointer	A variable that stores an address
<code>Node*</code>	Pointer to a node
<code>Node**</code>	Pointer to an array of node pointers
<code>nullptr</code>	Means no node exists there
Queue	First-in, first-out data structure
FIFO	First In, First Out
Sentinel value	A special value used as a signal
<code>-1</code>	Sentinel value meaning no child
Level-order construction	Building the tree level by level

39. Phase 3 Summary

By the end of Phase 3, you should know the following:

- A binary tree can be stored using an array.
- A binary tree can also be stored using linked nodes.
- Array representation stores nodes level by level.
- In this curriculum, array representation starts at index 1.
- If a node is at index i , its left child is at $2*i$.
- If a node is at index i , its right child is at $2*i + 1$.
- If a node is at index i , its parent is at $i/2$.
- Array representation is best for complete or almost-complete trees.

- Array representation can waste space for sparse or skewed trees.
- Linked representation uses pointers.
- A linked binary tree node has data, left child pointer, and right child pointer.
- `nullptr` means no child exists.
- A queue helps build a binary tree level by level.
- The queue should store `Node*`, not `int`.
- `-1` means no child and should not be created as a node.
- Newly created children must be pushed into the queue.

40. Final Beginner Memory Tricks

Remember these:

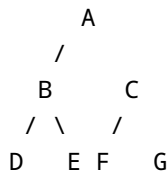
```
Array representation = positions
Linked representation = pointers
Queue construction = build level by level
-1 = no child
nullptr = no node address
```

Another useful memory trick:

```
Array asks: Where is the child position?
Linked asks: Where does the pointer point?
Queue asks: Which node needs children next?
```

41. Practice Questions

Use this tree:



Answer these questions:

1. If A is at index 1, what index is B?
2. If A is at index 1, what index is C?

3. If B is at index 2, what index is D?
 4. If B is at index 2, what index is E?
 5. If C is at index 3, what index is F?
 6. If C is at index 3, what index is G?
 7. What is the parent index of G?
 8. What is the parent node of G?
 9. What are the three fields inside a linked binary tree node?
 10. Why should child pointers be initialized to `nullptr`?
 11. Why do we use a queue to build a tree level by level?
 12. Why should the queue store `Node*` instead of `int`?
 13. What does `-1` mean during construction?
 14. What happens if we forget to push a newly created child into the queue?
 15. Which representation is usually better for sparse trees: array or linked?
-

42. Practice Answers

Answer 1

B is the left child of A.

$$\text{Left child} = 2 * 1 = 2$$

So B is at index 2.

Answer 2

C is the right child of A.

$$\text{Right child} = 2 * 1 + 1 = 3$$

So C is at index 3.

Answer 3

D is the left child of B.

B is at index 2.

$$\text{Left child} = 2 * 2 = 4$$

So D is at index 4.

Answer 4

E is the right child of B.

B is at index 2.

$$\text{Right child} = 2 * 2 + 1 = 5$$

So E is at index 5.

Answer 5

F is the left child of C.

C is at index 3.

$$\text{Left child} = 2 * 3 = 6$$

So F is at index 6.

Answer 6

G is the right child of C.

C is at index 3.

$$\text{Right child} = 2 * 3 + 1 = 7$$

So G is at index 7.

Answer 7

G is at index 7.

$$\text{Parent} = 7 / 2 = 3$$

So the parent index of G is 3.

Answer 8

Index 3 contains C.

So the parent node of G is C.

Answer 9

A linked binary tree node contains:

1. Data
2. Left child pointer
3. Right child pointer

Example:

```
int data;  
Node* lchild;  
Node* rchild;
```

Answer 10

Pointers should be initialized to `nullptr` so they do not contain garbage addresses.

`nullptr` means no child exists.

Answer 11

We use a queue because tree construction happens level by level.

The first node created should be the first node whose children are processed.

That is exactly how a queue works.

Answer 12

The queue should store `Node*` because we need actual node addresses.

Only node addresses allow us to attach children.

Example:

```
p->lchild = new Node(40);
```

This requires `p` to be a node pointer.

Answer 13

`-1` means no child exists.

It is not created as a node.

Answer 14

If we forget to push a newly created child into the queue, the program will never process that child.

So the child will never get its own children.

Answer 15

Linked representation is usually better for sparse trees because it does not require many empty array positions.

43. Final Checklist Before Moving to Phase 4

Before moving to Phase 4, make sure you can answer these:

- What is array representation?
- Why does the array start at index 1?
- What is the formula for the left child?
- What is the formula for the right child?
- What is the formula for the parent?
- When can array representation waste space?
- What is linked representation?
- What does `Node*` mean?
- What does `nullptr` mean?
- Why does a binary tree node need two child pointers?
- Why do we use a queue to create a tree?
- Why does the queue store node addresses?
- What does `-1` mean in input?
- How does level-order construction work?

If you can answer these questions, you understand Phase 3 well.

44. Final Phase 3 Memory Statement

Phase 3 is mainly about this:

A binary tree can be stored using array positions or linked pointers.

Array representation uses formulas:

```
left = 2*i
right = 2*i + 1
parent = i/2
```

Linked representation uses pointers:

```
Node* lchild;
Node* rchild;
```

Queue-based construction builds the tree level by level:


```
Create root -> push root -> process one node -> create children -> push children  
-> repeat
```

Once this is clear, Phase 4 becomes easier because traversal means visiting every node in the tree you now know how to build.